

PRISM: Paradigm-Guided Reasoning with Iterative Solver Memory

Supplementary Materials

Anonymous submission

Appendix A: Complete Algorithms

Algorithm 1: Offline Paradigm Distillation

Algorithm 2: Online Paradigm-Guided Solving

Appendix B: Theoretical Foundations

Soundness of Paradigm Verification

Claim: If paradigm P passes the triple verification tests (Soundness $\geq \tau_s$, Effect, Precision $\geq \tau_p$), then applying P to any solver state satisfying trigger conditions $\mathcal{Q}_P$ preserves Z3 satisfiability with high probability.

Proof Sketch: The Soundness test directly verifies that $\text{Z3-SAT}(C_i \cup \{\text{op}(P)\})$ holds for representative constraint sets. If the paradigm passes with score $\geq \tau_s = 0.90$, it is guaranteed satisfiable in at least 90% of sampled trigger-matching scenarios.

The Effect test provides two guarantees: (a) *non-contradiction* — the operation is consistent with the pre-condition ($\phi_{\text{pre}} \cup \{\text{op}(P)\}$ is SAT); (b) *non-vacuousness* — the pre-condition is satisfiable on its own, confirming the operation adds new information that is not already forced by ϕ_{pre} alone. A paradigm that fails non-vacuousness is trivially true under the pre-condition and provides no reasoning value.

The Precision test estimates trigger selectivity against a representative pool spanning ZebraLogic-domain patterns (position, binding, adjacency, exclusion) and generic arithmetic constraints. A paradigm that fires on most pool samples would produce unhelpful guidance across unrelated solver states.

Correctness of UNSAT Core Attribution

Claim: The two-level causal attribution rule correctly distinguishes NEW_ASSERTION (current action caused contradiction) from LEGACY_ERROR (contradiction predates current action), and the secondary classification further refines NEW_ASSERTION into a structured *ErrorType*.

Proof Sketch (Primary Attribution): Let a_t be the constraint string produced by the current repair and U_t be the UNSAT core extracted after applying a_t . By the definition of UNSAT core, U_t is a minimal subset of constraints whose conjunction is unsatisfiable.

- If $a_t \in U_t$: a_t is *necessary* for unsatisfiability. Removing a_t from the constraint set would restore satisfiability (by minimality of U_t). Hence a_t directly caused the contradiction: NEW_ASSERTION.
- If $a_t \notin U_t$: a_t is not required for the unsatisfiable subset. Unsatisfiability exists entirely within the prior constraint set, and a_t merely exposed a latent contradiction: LEGACY_ERROR.

This primary distinction is mechanically sound: Z3’s `unsat_core()` guarantees that each returned constraint is necessary for unsatisfiability (i.e., removing it yields a satisfiable set).

Secondary Classification: When primary attribution yields NEW_ASSERTION, syntactic heuristics applied to a_t further distinguish:

- **SemanticFlip** — contradictory comparison operators co-occurring in a_t indicate a constraint direction error (e.g. $x \geq k \wedge x \leq k - 1$).
- **ScopeError** — references to undefined variables indicate a binding/scoping issue introduced by the LLM formalisation.
- **OverConstraint** — default case: the new assertion legally tightens the domain beyond satisfiability.

The *ErrorType* label routes the next repair to the appropriate fix class, enabling semantically-targeted rather than random strategy selection.

Appendix C: Hyperparameter Analysis

Table C1 summarizes default hyperparameter values and their sensitivity ranges determined via grid search on ZebraLogic development set (100 puzzles).

Impact legend: High = substantial accuracy/efficiency change; Medium = noticeable but manageable; Low = minimal practical effect.

Appendix D: Case Studies

Case Study 1: Successful Paradigm Extraction

Scenario: 5×5 Logical Grid Puzzle with "binding" and "adjacency" constraints.

KDP Cluster: 12 KDPs sharing constraint signature {direct_binding, adjacent_left}, step type "CHAIN_PROPAGATION".

Algorithm 1 PRISM-Offline(Puzzles, LLM, Z3, θ , $m_{\min}$, τ_s , τ_e , τ_p)

```

1:  $T_{\text{all}} \leftarrow \emptyset$  {Initialize trajectory pool}
2: for each  $P \in \text{Puzzles}$  do
3:   for  $r = 1, 2, 3$  do
4:      $T \leftarrow \text{LLM-Z3-Solve}(P, \text{temperature} = 0.7)$ 
5:     if  $T.\text{outcome} == \text{SAT}$  then
6:        $T_{\text{all}} \leftarrow T_{\text{all}} \cup \{T\}$ 
7:     end if
8:   end for
9: end for
10: {Extract Key Decision Points}
11:  $\text{KDP}_{\text{all}} \leftarrow \emptyset$ 
12: for each  $T \in T_{\text{all}}$  do
13:   for each step  $(S_{t-1}, a_t, S_t) \in T$  do
14:     if  $\text{IsKDP}(S_{t-1}, a_t, S_t)$  then
15:        $\text{KDP}_{\text{all}} \leftarrow \text{KDP}_{\text{all}} \cup \{\text{KDP}_i\}$ 
16:     end if
17:   end for
18: end for
19: {Cluster KDPs}
20:  $\text{Clusters} \leftarrow \text{AgglomerativeCluster}(\text{KDP}_{\text{all}}, \theta, \text{metric}=\text{cosine})$ 
21:  $\text{Clusters} \leftarrow \{C : |C| \geq m_{\min}\}$  {Filter by support}
22: {Abstract paradigms and verify}
23:  $\mathcal{L} \leftarrow \emptyset$ 
24: for each cluster  $C_k$  do
25:    $\text{Reps} \leftarrow \text{SampleRepresentatives}(C_k, n = 10)$ 
26:    $\hat{P}_k \leftarrow \text{LLM-Abstract}(\text{Reps})$ 
27:   if  $\hat{P}_k \neq \text{NULL}$  then
28:      $\text{soundness} \leftarrow \text{CheckSoundness}(\hat{P}_k, \tau_s)$ 
29:     if  $\text{soundness} \geq \tau_s$  then
30:        $\text{effect} \leftarrow \text{CheckEffect}(\hat{P}_k, \tau_e)$ 
31:       if  $\text{effect} \geq \tau_e$  then
32:          $\text{precision} \leftarrow \text{CheckPrecision}(\hat{P}_k, \tau_p)$ 
33:         if  $\text{precision} \geq \tau_p$  then
34:            $\hat{P}_k.\text{confidence} \leftarrow 0.5 \cdot \text{soundness} + 0.3 \cdot \text{effect}$ 
35:            $\mathcal{L} \leftarrow \mathcal{L} \cup \{\hat{P}_k\}$ 
36:         end if
37:       end if
38:     end if
39:   end if
40: end for
41: return  $\mathcal{L}$ 

```

Algorithm 2 PRISM-Online(P^* , $\mathcal{L}$, LLM, Z3, τ_{stag} , τ_{loop})

```

1:  $\mathcal{M} \leftarrow \emptyset$  {Initialize repair memory}
2:  $\text{Solver} \leftarrow \text{Z3SolverWrapper}()$ 
3:  $\text{Constraints} \leftarrow \text{LLM-Translate}(P^*)$ 
4: for each  $c \in \text{Constraints}$  do
5:    $\text{Solver.AddConstraint}(c)$ 
6: end for
7:  $\text{result} \leftarrow \text{Solver.Check}()$ 
8: if  $\text{result} == \text{SAT}$  then
9:   return Success
10: end if
11: {Repair loop}
12: for  $t = 1$  to  $\text{MaxRepairRounds}$  do
13:    $\sigma(S_t) \leftarrow \text{ExtractConstraintTypes}(S_t)$ 
14:    $\mathcal{L}_1 \leftarrow \text{RetrieveLayer1}(\sigma(S_t), \mathcal{L})$ 
15:    $\mathcal{L}_2 \leftarrow \text{RetrieveLayer2}(\mathcal{L}_1, S_t)$  {if  $|\mathcal{L}_1| \geq 2$ }
16:    $\mathcal{L}_{\text{inject}} \leftarrow \text{ConsistencyPreCheck}(\mathcal{L}_2, S_t)$ 
17:    $\text{hint} \leftarrow \text{FormatParadigmHint}(\mathcal{L}_{\text{inject}})$ 
18:    $\text{history} \leftarrow \mathcal{M}.\text{GetHistorySummary}()$ 
19:    $\text{action} \leftarrow \text{LLM-Repair}(\text{constraints}, \text{unsat\_core}, \text{history}, \text{hint})$ 
20:    $S_t \leftarrow \text{Solver.AddAndCheck}(\text{action})$ 
21:   if  $S_t.\text{result} == \text{SAT}$  then
22:     return Success
23:   else
24:      $\text{unsat\_core} \leftarrow \text{Solver.GetUnsatCore}()$ 
25:      $\text{record} \leftarrow \text{CreateRepairRecord}(t, \text{action}, \text{unsat\_core})$ 
26:      $\mathcal{M}.\text{Append}(\text{record})$ 
27:     if  $\mathcal{M}.\text{DetectStagnation}()$  OR  $\mathcal{M}.\text{DetectLoop}(\text{action})$  then
28:        $\text{level} \leftarrow \text{DetermineEscalationLevel}(\mathcal{M})$ 
29:        $\text{ExecuteStrategyEscalation}(\text{level}, S_t)$ 
30:     end if
31:   end if
32: end for
33: return Failure

```

Table 1: Hyperparameter Sensitivity Analysis

Parameter	Default	Range	Impact
Cluster threshold (θ)	0.25	[0.15, 0.35]	Med.
Min support ($m_{\min}$)	5	[3, 10]	Low
Soundness (τ_s)	0.90	[0.85, 0.95]	High
Effect (τ_e)	0.80	[0.75, 0.90]	Med.
Precision (τ_p)	0.80	[0.75, 0.90]	Med.
Stag. Jaccard (τ_{stag})	0.75	[0.60, 0.85]	High
Loop cosine (τ_{loop})	0.90	[0.85, 0.95]	Med.
Min confidence	0.50	[0.40, 0.65]	Med.
Top-K (k)	3	[1, 5]	Low

Representative Steps:

- Step A: "If house A has color X, then person B cannot have color X"
- Step B: "House 1 has nationality Norwegian; therefore all other houses cannot be Norwegian"
- Step C: "The blue house and the person who drinks wine are adjacent"

LLM Abstraction Output:

- **Paradigm:** P1 (Direct Assignment Propagation)
- **Trigger:** constraint type = `direct.binding`; state feature = domain size ≥ 2 for target.
- **Operation:** When entity X has unique property V, remove V from all other entities in the category.
- **Template:** `domain[X] := {V}; for all Y != X: domain[Y] -= V.`
- **Precondition:** `Exists c in C: implies(entity_X, property_V).`
- **Postcondition:** `entity_X == property_V` and all other entities do not take `property_V`.
- **Scope:** 3×5 to 6×6 logical grids, all difficulty levels.

Verification Results: - Soundness: 0.94 (47/50 trials successful) - Effect: 0.86 (43/50 trials reduced domains) - Precision: 0.85 (17/20 negative cases correctly UNSAT) - **Status:** Accepted into Paradigm Library

Case Study 2: Repair Escalation

Scenario: 6×6 puzzle where initial translation generates over-constrained formula.

Repair Sequence:

1. **L1 Attempt** (Switch Target): Modify different constraint from UNSAT core $\rightarrow$ still UNSAT.
2. **L2 Attempt** (Switch Type): Change from "relax-bound" to "split-constraint" $\rightarrow$ still UNSAT.
3. **Loop Detection Triggers:** Cosine similarity of recent repairs = 0.92 (> 0.90 threshold).
4. **L3 Execution** (Revert + Retry): Restore solver to SAT checkpoint; explore alternative inference path $\rightarrow$ SAT.

Analysis: Without adaptive escalation, the system would exhaust repair budget (8 rounds) without solution. L3 strategy enables backtracking to search alternative branches.

Appendix E: Computational Complexity**Offline Phase**

- **Trajectory Collection:** $O(N \cdot r \cdot T)$ where N = puzzles, r = runs per puzzle, T = steps per trajectory. For $N = 600$, $r = 3$, avg $T = 20$: $\approx 36,000$ solver steps.
 - **KDP Extraction:** $O(N \cdot r \cdot T)$ (single pass over trajectories). Typically ≈ 6000 KDPs extracted.
 - **Clustering:** $O(K^2 \log K)$ where $K = 6000$ KDPs, metric = cosine distance. Single-linkage agglomerative: $\approx 150M$ pairwise operations.
 - **LLM Abstraction:** $O(C \cdot \text{LLM-cost})$ where $C \approx 50$ clusters. LLM cost dominated by token processing (typically 2-5 seconds per paradigm).
 - **Z3 Verification:** $O(P \cdot 90)$ where $P \approx 60$ candidate paradigms, 90 = trials per verification. Each trial: $\approx 100ms$ Z3 execution.
- Total Offline Time:** 24-48 hours (dominated by LLM abstraction and Z3 verification, parallelizable).

Online Phase

- **Constraint Feature Extraction:** $O(|C_t|)$ per step. $|C_t| \approx 30 - 80$ constraints.
 - **Layer-1 Retrieval:** $O(|\mathcal{L}|) = O(35)$ set-membership checks.
 - **Layer-2 Semantic Matching:** $O(k \cdot \text{LLM-cost})$ where $k \leq 5$. Single LLM call if $k \geq 2$.
 - **Z3 Consistency Check:** $O(k \cdot \text{Z3-cost})$, approximately 2.5 seconds for $k = 5$ and 500ms per check.
 - **Z3 Verification + UNSAT Core:** $O(\text{Z3-cost})$, approximately 500ms per step.
 - **Stagnation/Loop Detection:** $O(L)$ for recent repairs, typically $L \leq 8$. Jaccard computation is $O(|\text{core}|^2)$, approximately $O(80)$.
- Per-Puzzle Online Time:** 10-30 seconds (depends on puzzle difficulty and repair rounds).

Appendix F: Experimental Details**Baseline Implementation Details**

- **Logic-LM:** Original implementation from Pan et al. with CSP-specific prompting.
- **ExpeL (CSP version):** Experience replay memory adapted from Ouyang et al., storing recent successful trajectories.
- **Neural-Symbolic Baseline:** LLM+Z3 with 5-round repair budget, no paradigm library or memory.
- **Verified Few-Shot:** Direct injection of 5-10 manually verified constraint-solving patterns, no learning.

Data Split

- **Training Set:** 600 puzzles (200 each of 4×4 , 5×5 , 6×6)
- **Development Set:** 100 puzzles (even split across scales)
- **Test Set:** 300 puzzles (even split across scales) — held-out, used only for final evaluation

Evaluation Metrics

1. **Solving Accuracy:** Percentage of puzzles solved correctly within repair budget.
2. **Average LLM Calls:** Mean number of API calls per puzzle.
3. **Average Repair Rounds:** Mean constraint-repair iterations before solution or failure.
4. **Paradigm Trigger Rate:** Fraction of solving steps where ≥ 1 paradigm retrieves.
5. **Paradigm Hit Rate:** Among triggered paradigms, fraction correctly applied (heuristic: improves UNSAT core or achieves SAT).
6. **Stagnation Frequency:** Percentage of repair sequences where stagnation detection fires.
7. **Transfer Rate (L1):** Cross-scale accuracy (e.g., paradigms trained on $4 \times 4 + 5 \times 5$ applied to 6×6).
8. **Transfer Rate (L2):** Cross-domain accuracy (e.g., paradigms from logical grids applied to graph coloring).

```
max_repair_rounds: 8
retrieval:
  layer2_enabled: true
  paradigm_top_k: 3
  min_confidence: 0.50
```

Appendix G: Implementation and Reproducibility

Code Availability

All code, trained paradigm libraries, and experimental logs are available at: [GitHub URL to be filled]

Reproducibility Requirements

- **Python Version:** 3.10+
- **Key Dependencies:**
 - z3-solver $\geq 4.12.0$
 - sentence-transformers (all-MiniLM-L6-v2)
 - scikit-learn (agglomerative clustering)
 - OpenAI API ≥ 0.27 or equivalent
- **Computational Requirements:**
 - Offline phase: 48 GPU-hours (for LLM abstraction and verification)
 - Online evaluation: 100 CPU-hours (batch evaluation on test set)
- **Random Seeds:** All experiments use seed=42 for reproducibility.

Configuration Files

Key configuration parameters (stored in 'config/default.yaml'):

```
clustering:
  theta: 0.25
  min_support: 5
verification:
  soundness_threshold: 0.90
  effect_threshold: 0.80
  precision_threshold: 0.80
repair:
  stagnation_jaccard: 0.75
  loop_cosine: 0.90
```